GENAI BASICS • LESSON 14 • MODULE 4: IDEA TO PRODUCT

# Rapid AI app prototype

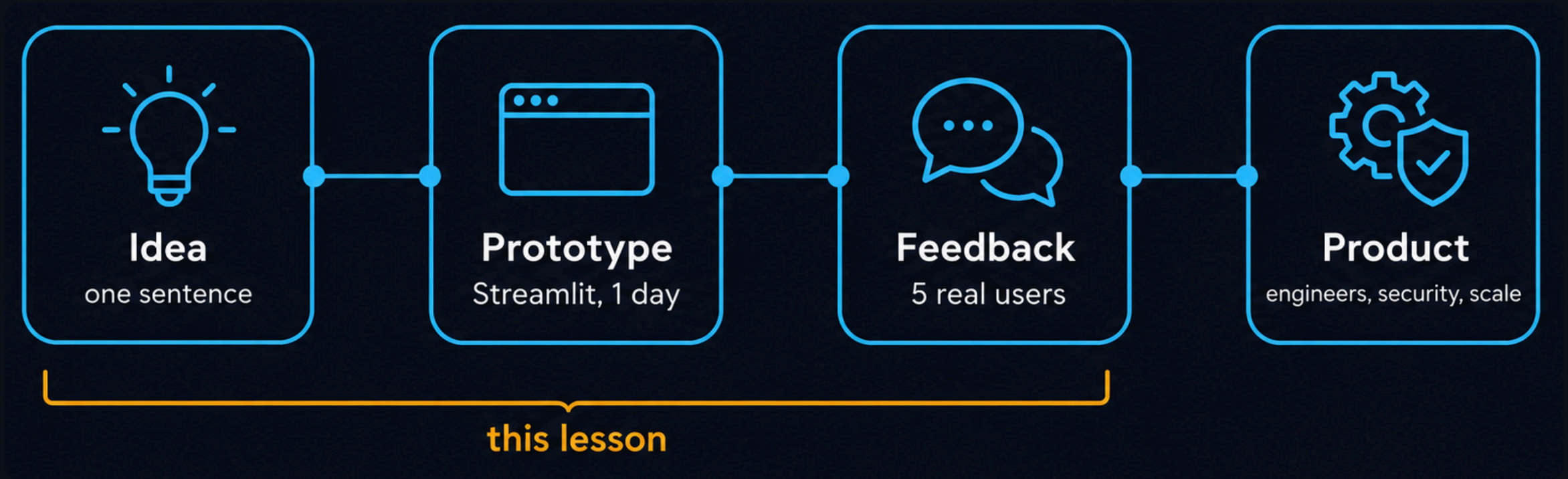
## Your own chat window with Streamlit in an hour — and when a no-code builder is enough

Streamlit, a coding agent, publishing, secrets. Google AI Studio Build, Claude Artifacts. Data as of October 2026.



WHY

# A prototype answers “what will it look like?” in a day, not a quarter

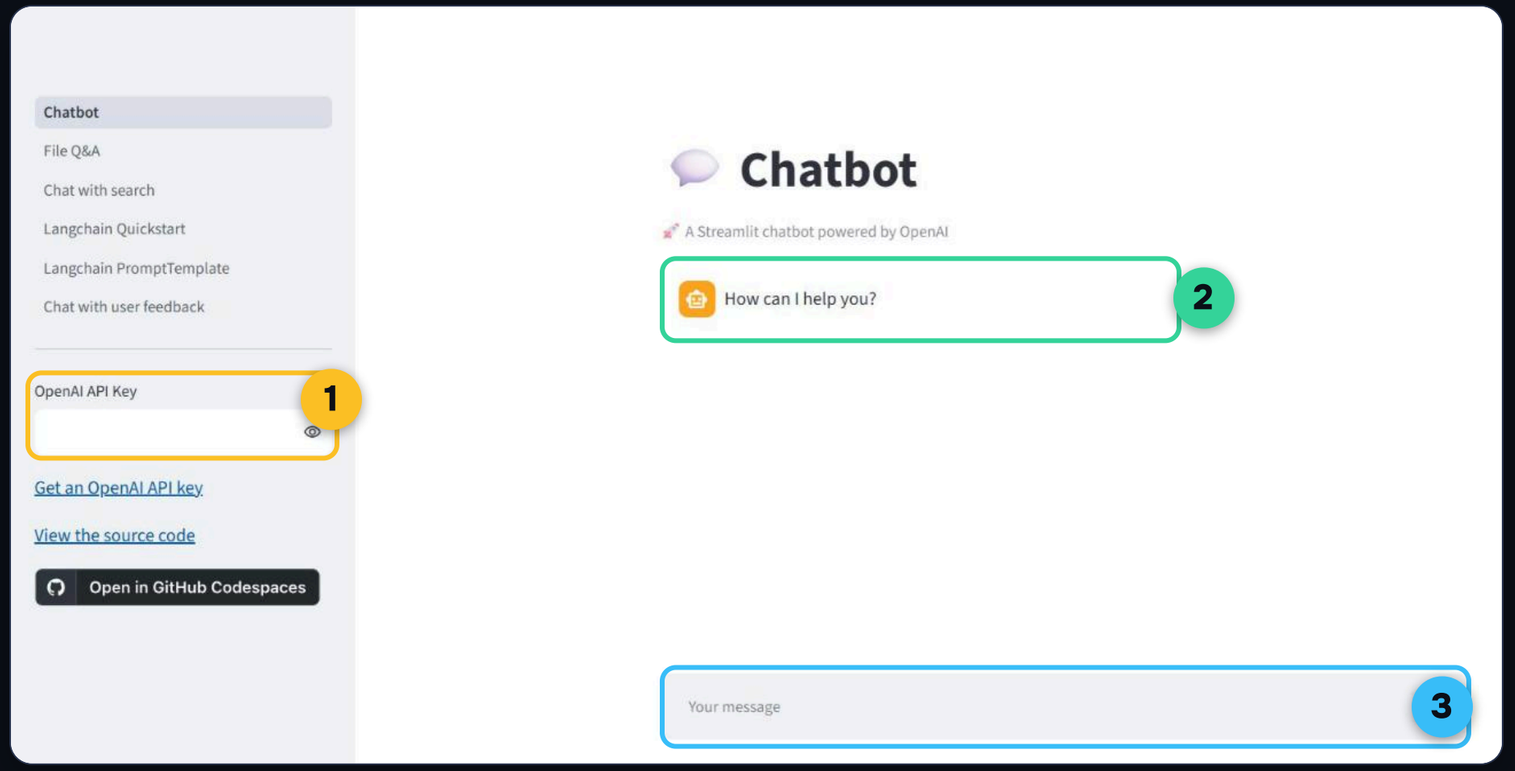



> A clickable prototype gathers requirements better than any meeting: people click, break it and say what they actually miss.

STREAMLIT

# Streamlit is a web page in Python: no HTML, no JavaScript

OFFICIAL EXAMPLE LLM-EXAMPLES.STREAMLIT.APP · CAPTURED 16.10.2026




1. **Sidebar**: a field for the API key
2. **Chat messages**
3. **Input box** at the bottom

> All of this is about 25 lines of Python. Free, open source (Apache-2.0), version 1.66.

CODE (REFERENCE)

# The whole chat is 12 lines: show history, take the question, ask the model, show the answer

APP.PY · RUN: STREAMLIT RUN APP.PY

```
import streamlit as st

st.title("Support assistant")

if "messages" not in st.session_state:
    st.session_state.messages = []          # chat memory

for m in st.session_state.messages:          # show history
    st.chat_message(m["role"]).write(m["content"])

if prompt := st.chat_input("Your question"):
    st.session_state.messages.append({"role": "user", "content": prompt})
    st.chat_message("user").write(prompt)
    answer = ask_llm(st.session_state.messages)   # the L03 API
    st.session_state.messages.append({"role": "assistant", "content": answer})
    st.chat_message("assistant").write(answer)
```

**session_state** — chat memory between clicks

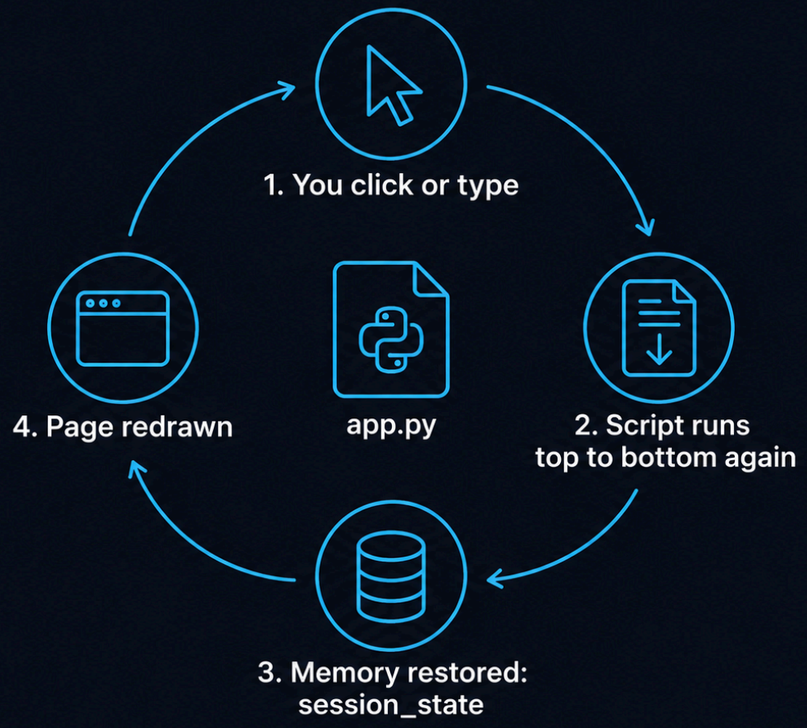
**for … chat_message** — show the whole history

**chat_input** — the input box at the bottom

**ask_llm** — your model call: Gemini, OpenAI, the local one from L10

HOW IT WORKS

# On every click Streamlit reruns the whole file — that is why memory is needed




Anything not stored in **session_state** disappears on the next click.

The most common beginner “bug” — “the chat gets wiped” — is not a bug, it is this loop.

THREE WORDS

# Memory, cache and secrets — three things not to mix up

**`session_state`**
one visitor’s memory: chat history, chosen settings

**`cache`**
a shared cache for everyone: a loaded file, a heavy calculation — not redone

**`secrets`**
API keys in .streamlit/secrets.toml, never in code

> A key in code = a key on GitHub (the site where code is stored) = a key for everyone. Add secrets.toml to **.gitignore** — the list of files that never go to GitHub.

WITH AN AGENT

# A coding agent writes the code; you set the task and check the result

PROMPT FOR CLAUDE CODE, CODEX OR ANTIGRAVITY

```
Build a Streamlit prototype: a chat assistant that answers staff from
faq.md.
1. Python 3.11+, a separate environment, pip install streamlit google-
genai.
2. First run streamlit skills to add Streamlit’s official skills.
3. Model Gemini 3.8 Flash, key from st.secrets; with no key, a demo mode
with a canned answer.
4. History in session_state, read faq.md with st.cache_data.
5. secrets.toml in .gitignore, plus a secrets.example.toml with a
placeholder.
6. Run streamlit run app.py and explain every part of the code to me.
```

**streamlit skills** — an official command: it gives the agent instructions on writing Streamlit properly (Claude Code, Codex, Cursor).

**A demo mode without a key** — the prototype can be shown at once, even without paying for an API.

The last step is a must: ask the agent to explain the code.

RUNNING IT

# Four steps: install, run, edit, show

**Install**
`pip install streamlit`
Python 3.11 or newer

→

**Run**
`streamlit run app.py`
opens localhost:8501

→

**Edit**
`save the file → Rerun`
or Always rerun — then automatic

→

**Show**
`Community Cloud or screen share`
slide 9

**localhost:8501** — the app runs on your computer. It is not on the internet; on an office network a colleague may open it via the Network URL shown in the terminal.

Changed text or code — saved — clicked Rerun (the button appears top right). Handy to edit together with a client.

PUBLISHING

# Where to show a prototype: your computer, the free Streamlit cloud or your company cloud

- **Your own computer**
  free; only you see it; share your screen to show
- **Streamlit Community Cloud**
  free, from a GitHub repo (a code folder); apps are public by default, only one private app
- **Your company**
  Snowflake, your own cloud — role-based access; engineers decide

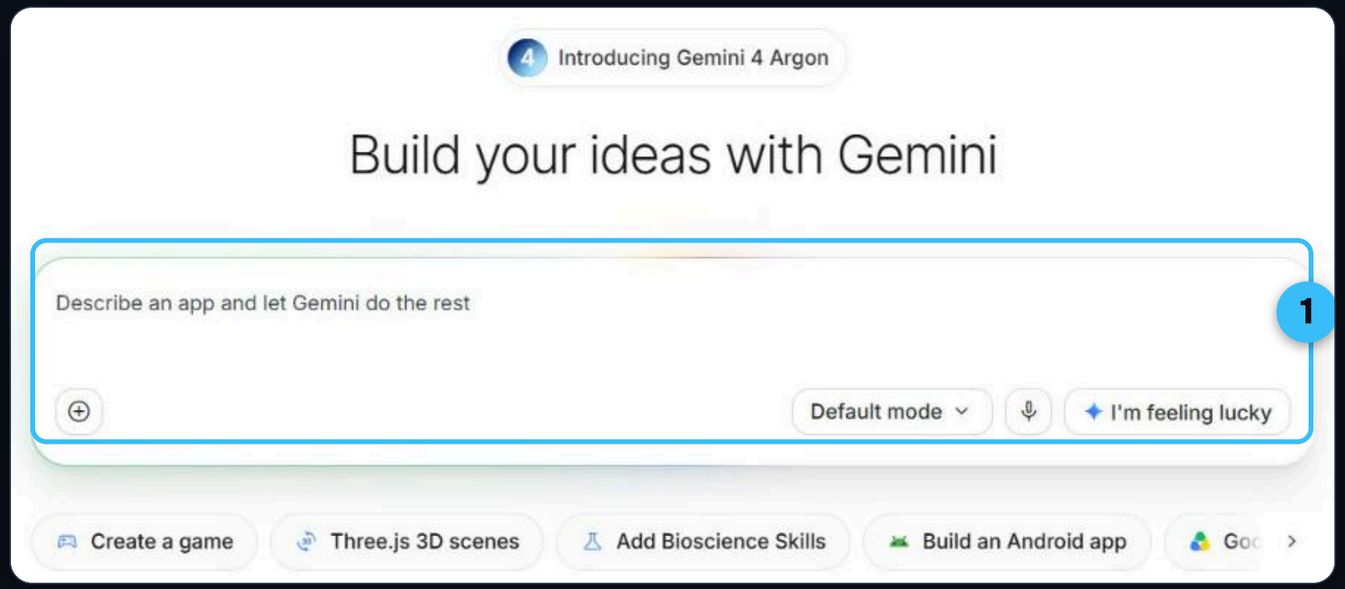
**A public link = public data and your API bill.** Anyone with the link spends your quota. A public prototype gets synthetic data only.

> A free app goes to sleep after 12 hours without visitors.

NO CODE

# Sometimes a builder is enough: an app from a description, no code at all

AISTUDIO.GOOGLE.COM → BUILD · CAPTURED 16.10.2026




**1 · Google AI Studio Build**
describe the app here — get a working one; private by default

**Claude Artifacts**
a small app right in the chat (L09)

**Lovable · Bolt · v0**
a whole site from a description; free — a small daily allowance

WHAT TO CHOOSE

# Choose by two questions: what you are showing and whether you will read code

| | Show an idea | Work with data |
|---|---|---|
| I don’t code | **AI Studio Build · Artifacts**<br>in 10 minutes, from a description | **Claude / ChatGPT + files**<br>analysis without an app (L08) |
| Ready to read code with an agent | **Streamlit**<br>your own UI, your own rules | **Streamlit + pandas**<br>pandas — a table library: charts, your own files |

BREAK

# 5 minutes: think which prototype you will show colleagues

# 05:00

Next: hands-on, security, the path to a product, homework.

HANDS-ON

# 10-minute hands-on: a helper prototype for your FAQ

**1** Idea
one sentence → **2** Data
FAQ text, synthetic → **3** Build
AI Studio Build or an agent → **4** Test
3 questions → **5** Show
Teams screen share

**No code:** AI Studio → Build → “Make a chat helper that answers from this FAQ” + paste 10 Q&As.

**With code** (only if Python and an agent are already installed): the slide 7 prompt; run streamlit run app.py.

SECURITY

# Four prototype rules: keys, data, money, sign-in

- **Keys**
  only in secrets, not in code and not in the chat with the agent
- **Data**
  a public prototype — synthetic data only
- **Money**
  a spending limit in the API console; a demo mode with no key
- **Sign-in**
  st.login when not everyone should see the prototype

A prototype is not a product. Until engineers and security review it, it does not touch real customer data.

NEXT

# What changes when a prototype becomes a product

| Prototype | → | Product |
| --- | --- | --- |
| one app.py file | → | code in a repo, reviews, tests |
| synthetic data | → | real data, access rights, logs |
| a developer’s key | → | a service key, limits, monitoring |
| works on my machine | → | works for everyone, 24/7 |

HOMEWORK

# Homework: a prototype and feedback from three real people

1. The prototype idea in one sentence: for whom and what it does
2. Build it: Streamlit with an agent, or AI Studio Build
3. Synthetic data only; the key in secrets
4. Show it to 3 colleagues and record their feedback
5. What you would change — 3 points

```
L14_homework/
├ README.md      # idea, conclusion
├ app.py or a link
├ faq.md         # synthetic
├ feedback.md    # 3 people’s feedback
└ demo.png
```

SUMMARY

# Lesson summary

**A prototype in a day**
beats a month of meetings

**Streamlit = Python + reruns**
memory in session_state

**Keys in secrets**
a public link = public data

**The agent writes, you check**
ask it to explain the code

> **Next, L15:** responsible AI, security and law — what you may and may not do with data and content.